



Kopilkin

Kopilkin C4 Architecture Explanation

Student Name

Mubarakov Islam

Group

11-314a

Assignment

3rd task series (2nd semester)

GitHub Repository

<https://github.com/iz-mv/Kopilkin>

Kopilkin C4 Architecture Explanation

Final C4 Diagrams for the Current Microservice Architecture

Student Name	Mubarakov Islam
Group	11-314a
Assignment	3rd task series (2nd semester)
Project	Kopilkin Personal Finance Application
GitHub Repository	https://github.com/iz-mv/Kopilkin

C4 Explanation

English Part

This document explains the final C4 diagrams for Kopilkin. The architecture now includes REST routing, two API Gateway instances, gRPC communication, Kafka event-driven processing, worker-service, PostgreSQL databases, Redis cache/locks/blacklist, MinIO object storage, Qdrant vector memory, Langfuse tracing and integration tests.

1. C4 Level 1 - System Context

This diagram shows Kopilkin as a complete system used by a personal finance user. It also shows the external development and inspection tools, the local Ollama runtime used for LLM calls, and the observability/memory infrastructure. The purpose of this level is to show system boundaries and external actors.

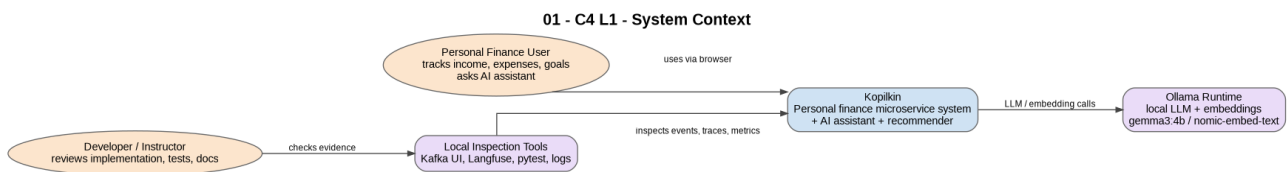


Figure - 1. C4 Level 1 - System Context

2. C4 Level 2 - Container Overview

This diagram shows the main runtime containers: frontend, Nginx load balancer, two API Gateway instances, core business services, AI services and infrastructure. It makes clear that the frontend calls only Nginx, while internal routing is handled by the API Gateway cluster.

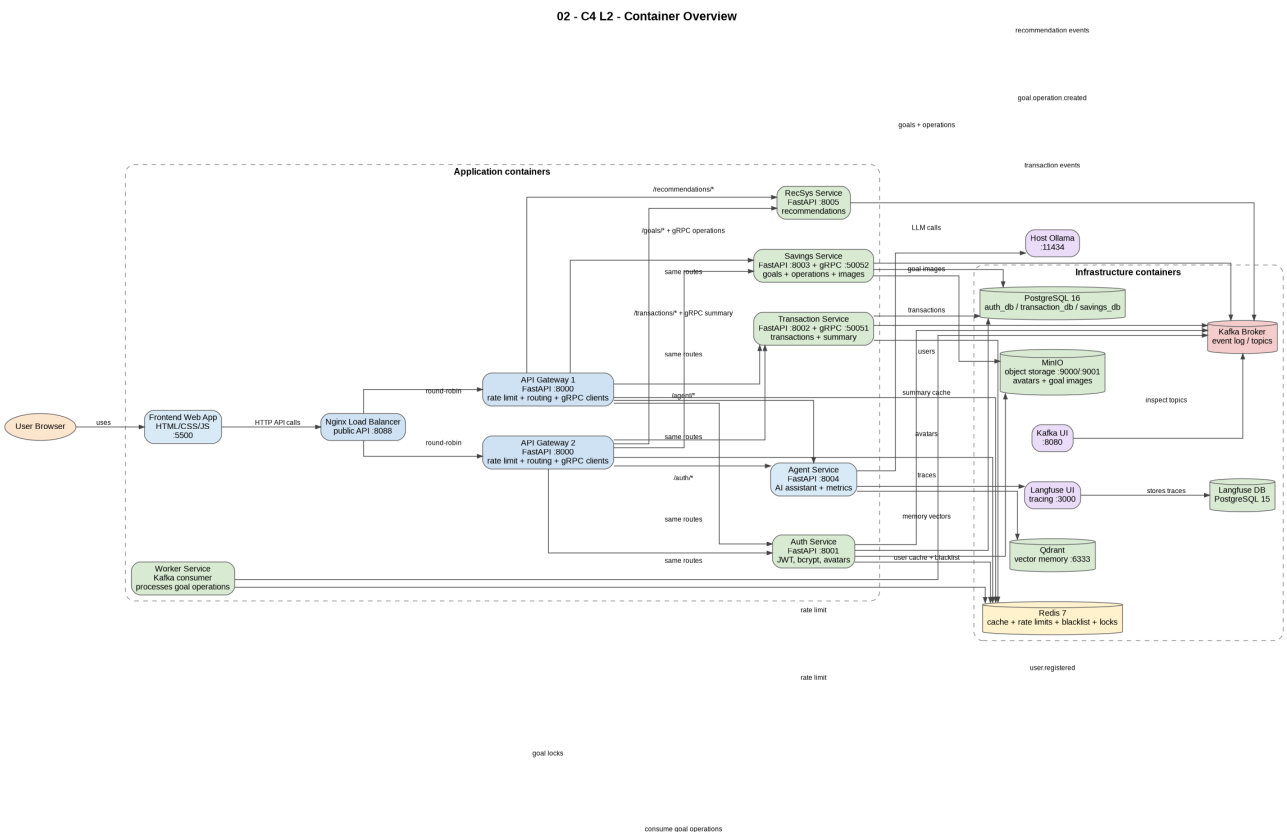


Figure - 2. C4 Level 2 - Container Overview

3. C4 Level 2 - Data, Events, Routing and Storage View

This diagram focuses on data ownership and infrastructure usage. It shows PostgreSQL databases, Redis cache and rate-limit keys, Kafka topics, MinIO object storage, Qdrant memory and Langfuse traces.

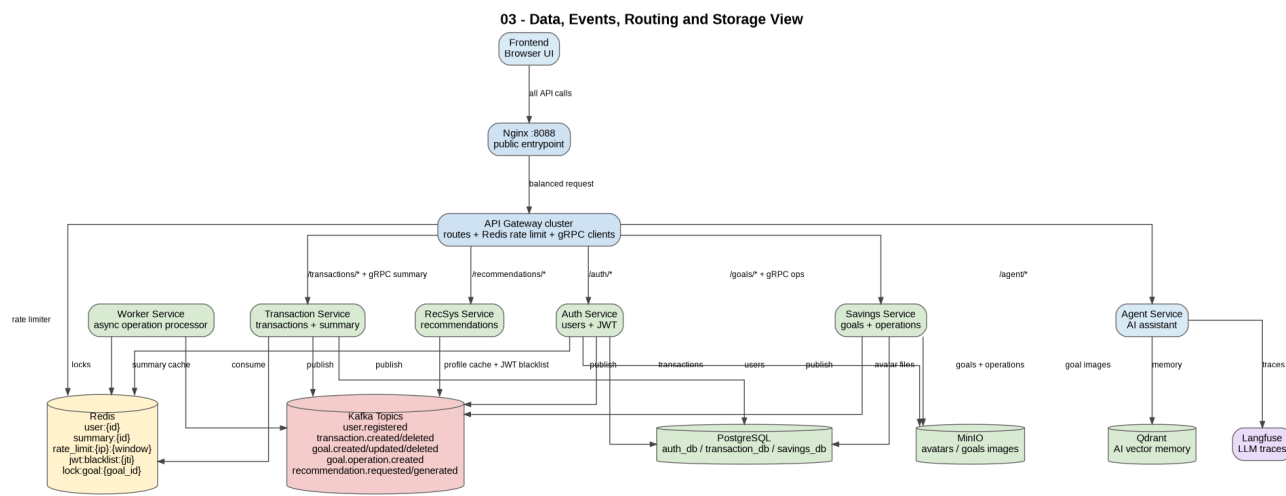


Figure - 3. C4 Level 2 - Data, Events, Routing and Storage View

4. C4 Level 3 - API Gateway Components

This diagram explains the internal structure of the API Gateway: CORS middleware, Redis rate limiter, route mapping layer, HTTP proxy client, gRPC client layer and environment-based service configuration. It is important because the gateway is the central entry point for frontend API calls.

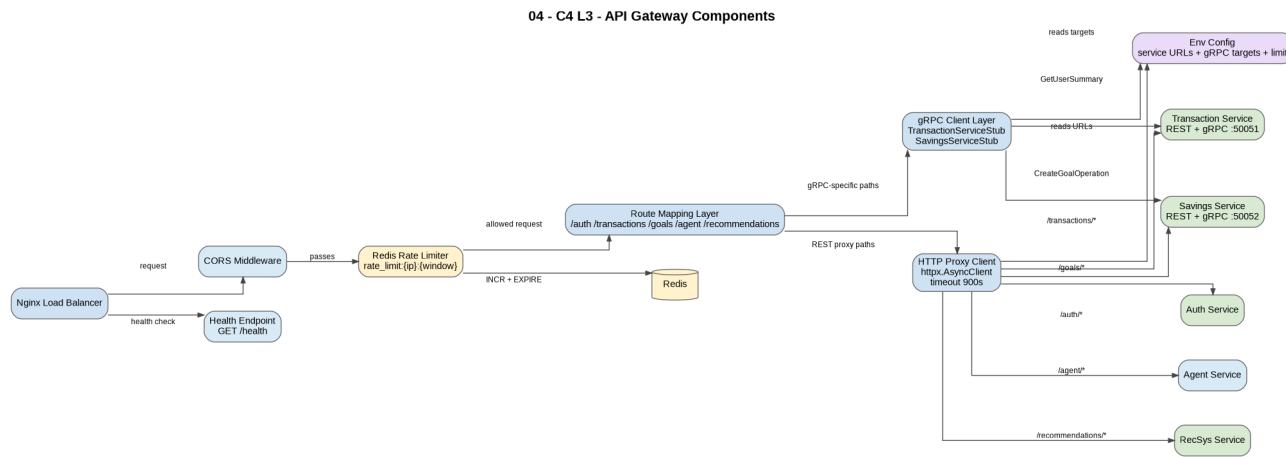


Figure - 4. C4 Level 3 - API Gateway Components

5. C4 Level 3 - Transaction Service Components

This diagram shows the Transaction Service internals: REST endpoints, Pydantic schemas, service logic, SQLAlchemy persistence, Redis summary cache, Kafka event publisher and gRPC server for GetUserSummary.

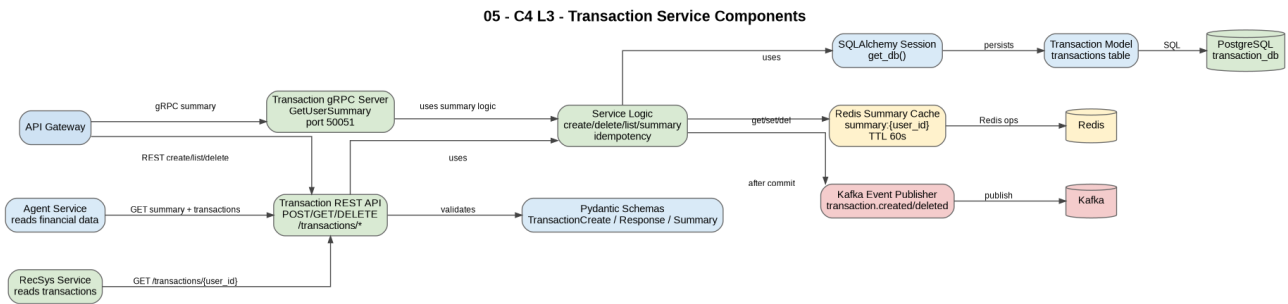


Figure - 5. C4 Level 3 - Transaction Service Components

6. C4 Level 3 - Savings and Worker Components

This diagram explains the final savings subsystem. Savings Service manages goals and creates GoalOperation records, while Worker Service consumes Kafka events, uses Redis locks, updates operation status and creates linked transactions.

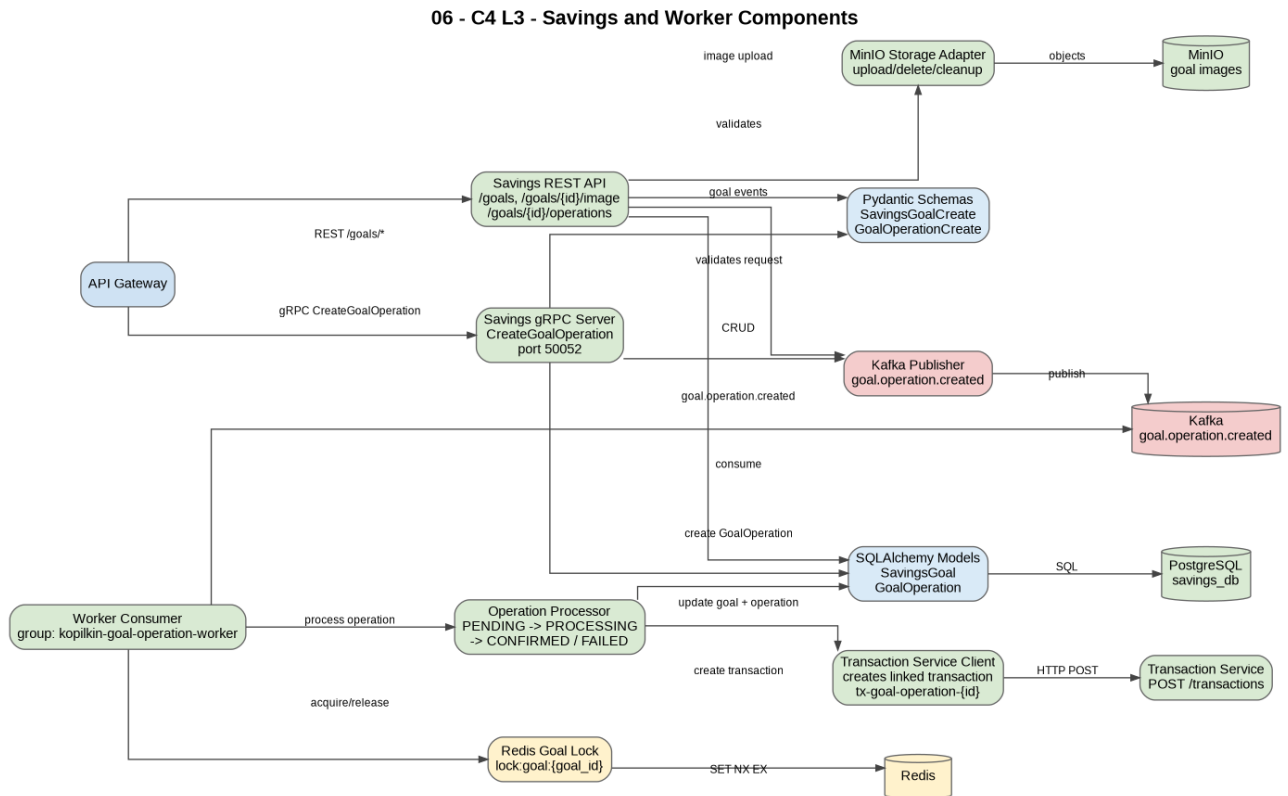


Figure - 6. C4 Level 3 - Savings and Worker Components

7. C4 Dynamic - Goal Operation via gRPC + Kafka + Worker

This dynamic diagram shows the most important runtime scenario: frontend sends a deposit/withdraw request, API Gateway calls Savings Service via gRPC, Savings Service publishes a Kafka event, Worker Service processes it, updates the goal and creates a related transaction.

07 - Dynamic Flow - Goal Operation via gRPC + Kafka + Worker

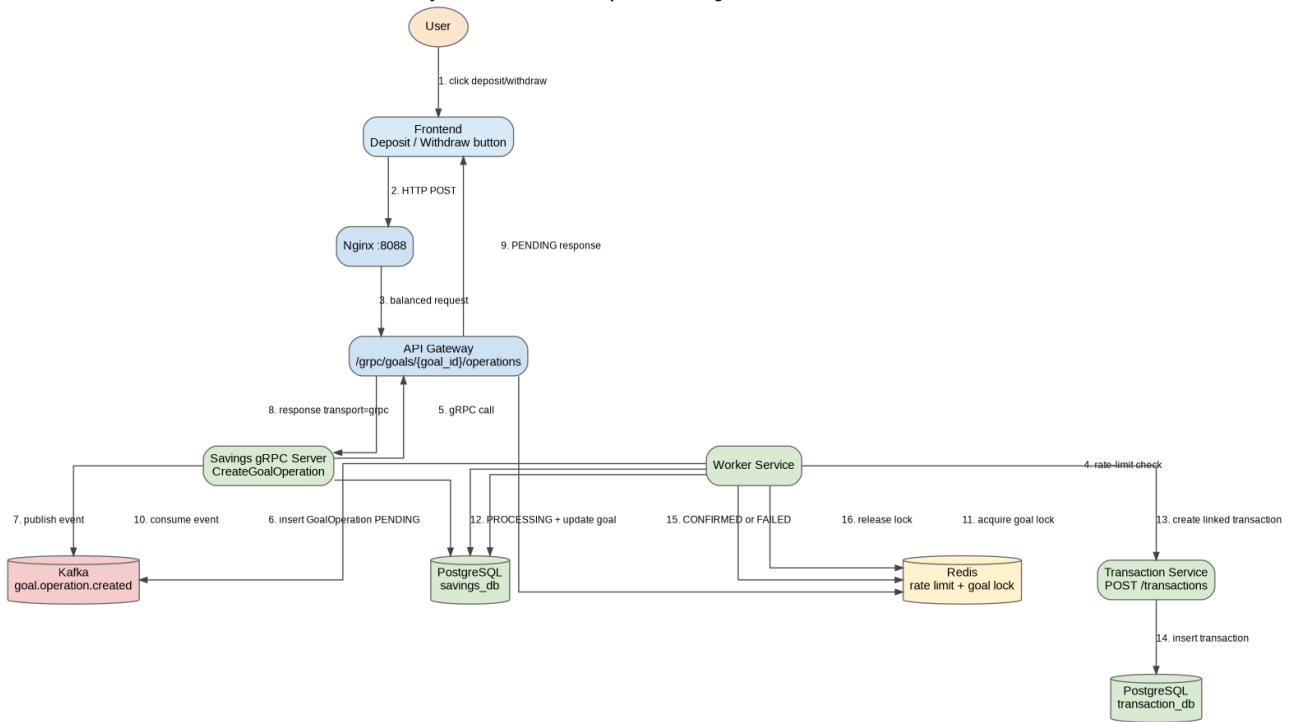


Figure - 7. C4 Dynamic - Goal Operation via gRPC + Kafka + Worker

8. Integration Test Coverage View

This supporting diagram is not a classic C4 diagram, but it explains what the integration tests prove: service health, JWT flow, MinIO uploads, gRPC calls, Kafka worker processing and recommendations.

08 - Integration Test Coverage

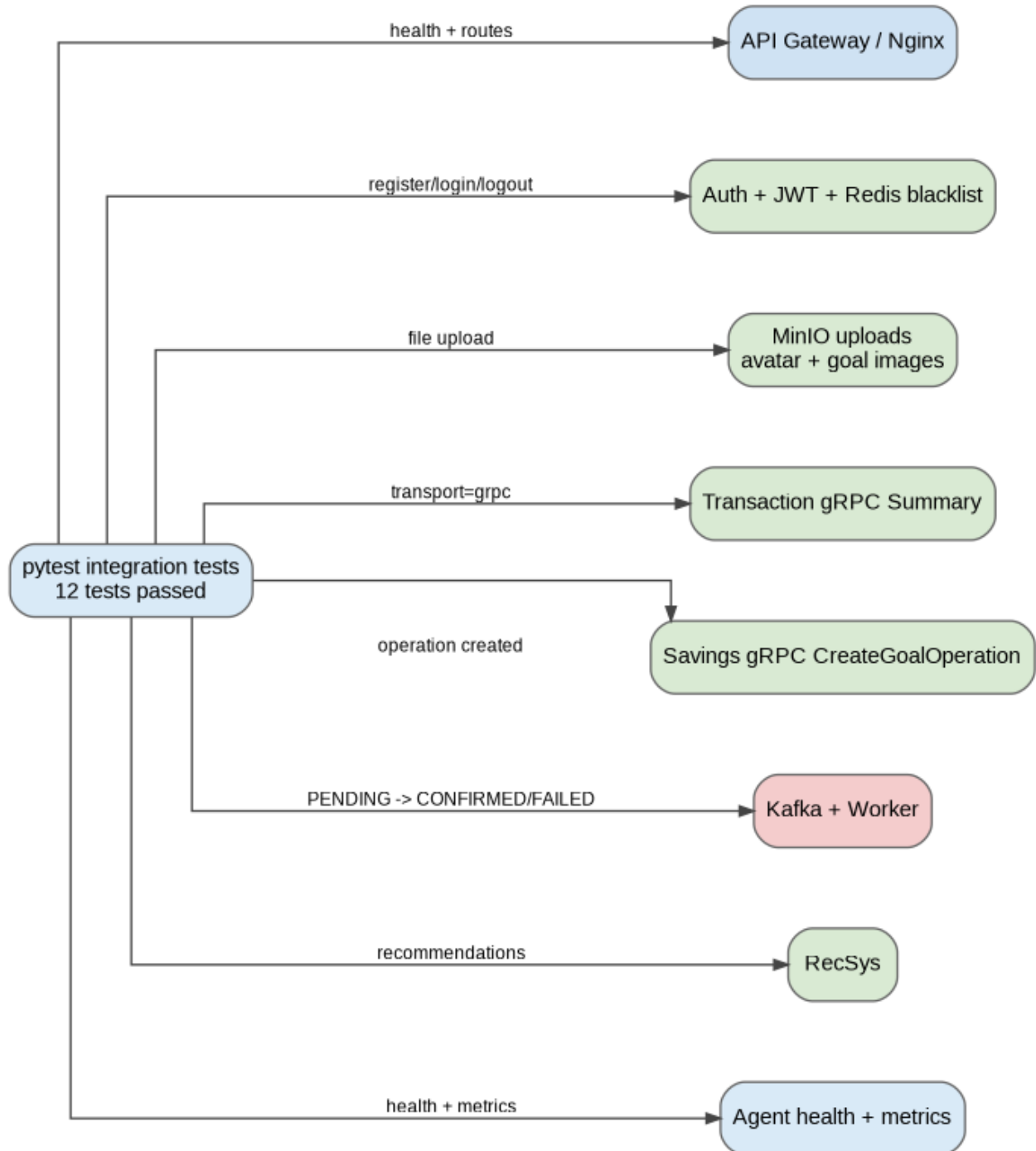


Figure - 8. Integration Test Coverage View

Русская часть

Этот документ объясняет финальные C4-диаграммы проекта Kopilkin. Архитектура сейчас включает REST routing, две API Gateway instances, gRPC communication, Kafka event-driven processing, worker-service, PostgreSQL databases, Redis cache/locks/blacklist, MinIO object storage, Qdrant vector memory, Langfuse tracing и integration tests.

1. C4 Level 1 - System Context

Диаграмма показывает Kopilkin как систему, которой пользуется пользователь personal finance app. Также показаны инструменты проверки, локальный Ollama runtime для LLM calls и infrastructure для observability/memory. Цель уровня - показать границы системы и внешних участников.

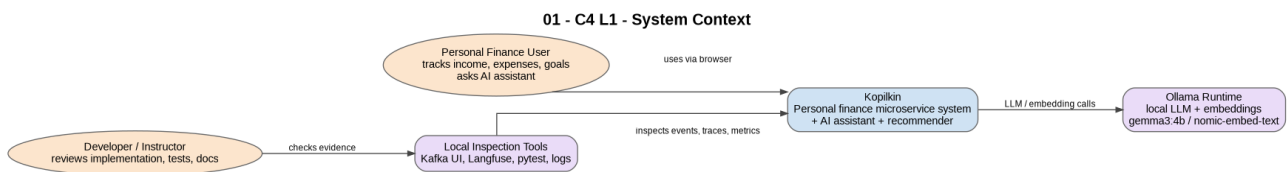


Рисунок - 1. C4 Level 1 - System Context

2. C4 Level 2 - Container Overview

Диаграмма показывает основные runtime containers: frontend, Nginx load balancer, две API Gateway instances, core business services, AI services и infrastructure. Видно, что frontend обращается только к Nginx, а внутренняя маршрутизация выполняется через API Gateway cluster.

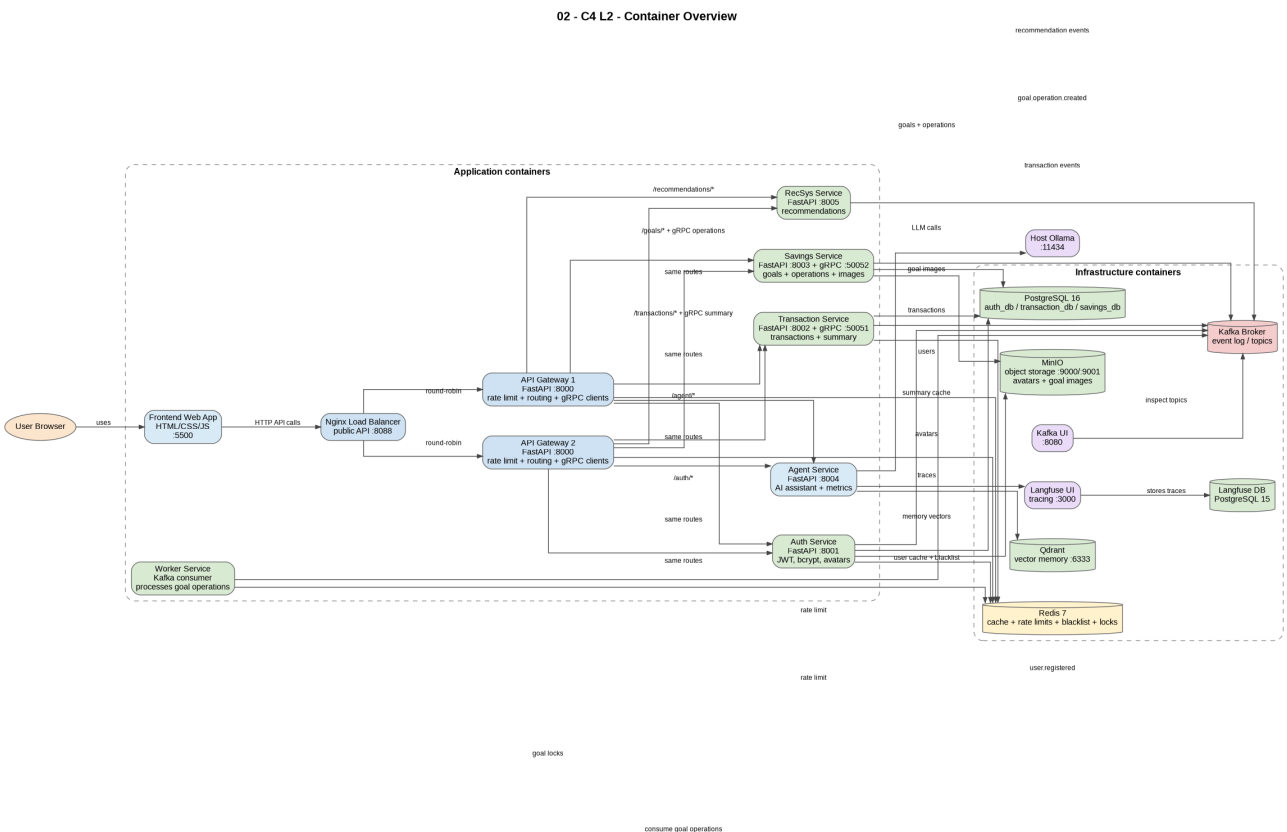


Рисунок - 2. C4 Level 2 - Container Overview

3. C4 Level 2 - Data, Events, Routing and Storage View

Диаграмма фокусируется на data ownership и infrastructure usage. Здесь показаны PostgreSQL databases, Redis cache and rate-limit keys, Kafka topics, MinIO object storage, Qdrant memory и Langfuse traces.

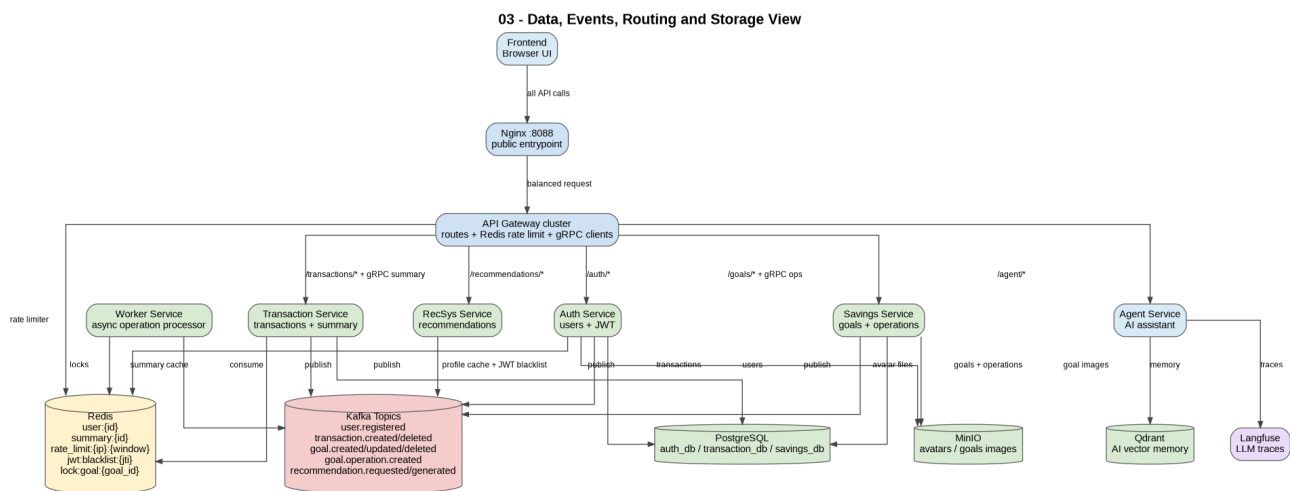


Рисунок - 3. C4 Level 2 - Data, Events, Routing and Storage View

4. C4 Level 3 - API Gateway Components

Диаграмма объясняет внутреннюю структуру API Gateway: CORS middleware, Redis rate limiter, route mapping layer, HTTP proxy client, gRPC client layer и env-based service configuration. Gateway является центральной точкой входа для API calls с frontend.

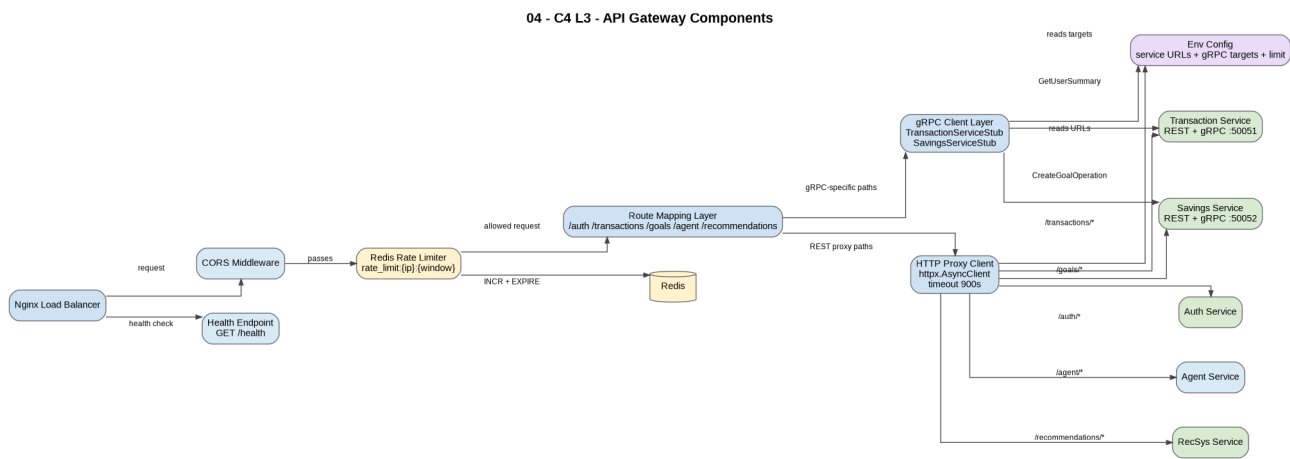


Рисунок - 4. C4 Level 3 - API Gateway Components

5. C4 Level 3 - Transaction Service Components

Диаграмма показывает внутренности Transaction Service: REST endpoints, Pydantic schemas, service logic, SQLAlchemy persistence, Redis summary cache, Kafka event publisher и gRPC server для GetUserSummary.

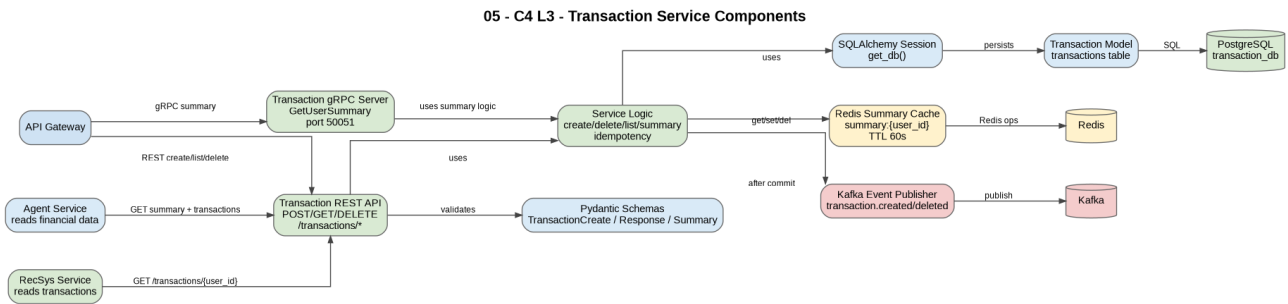


Рисунок - 5. C4 Level 3 - Transaction Service Components

6. C4 Level 3 - Savings and Worker Components

Диаграмма объясняет финальную подсистему savings. Savings Service управляет goals и создаёт GoalOperation records, а Worker Service читает Kafka events, использует Redis locks, обновляет status операции и создаёт связанные transactions.

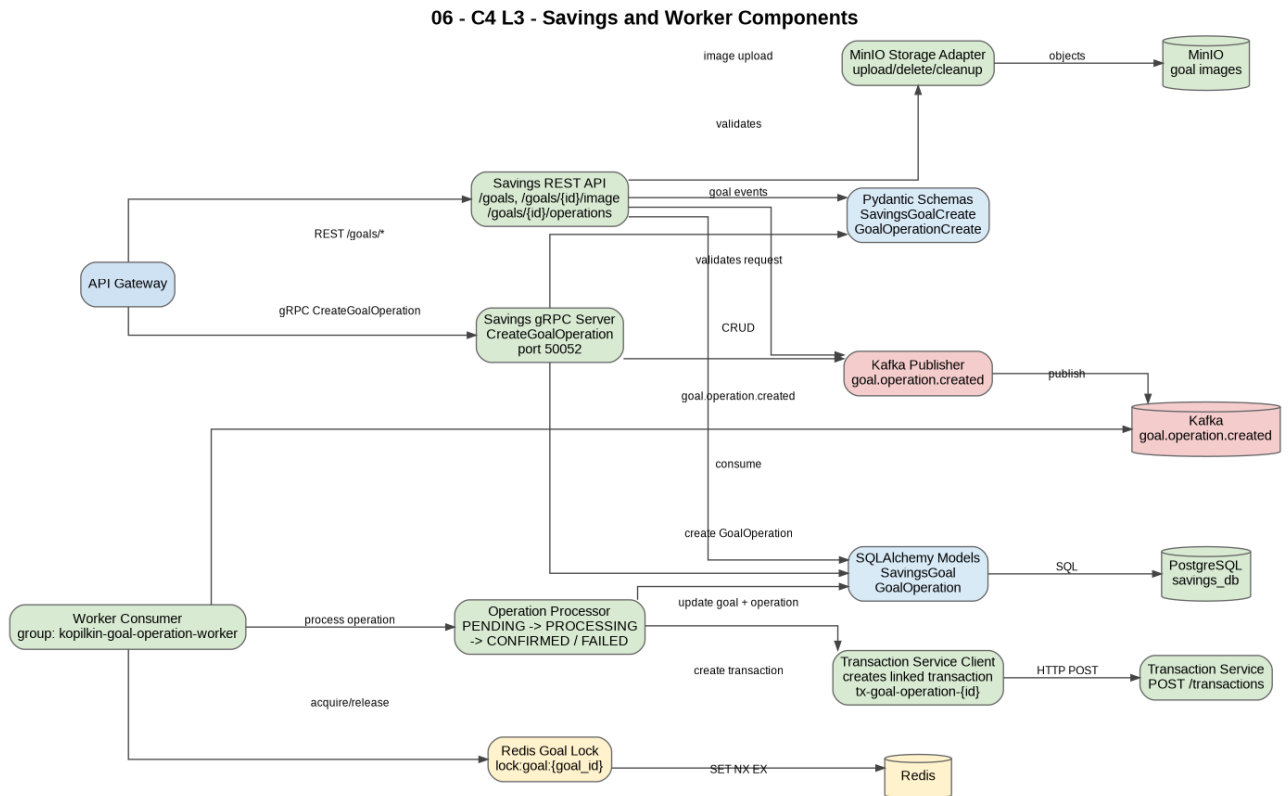


Рисунок - 6. C4 Level 3 - Savings and Worker Components

7. C4 Dynamic - Goal Operation via gRPC + Kafka + Worker

Эта dynamic diagram показывает главный runtime-сценарий: frontend отправляет deposit/withdraw request, API Gateway вызывает Savings Service через gRPC, Savings Service публикует Kafka event, Worker Service обрабатывает его, обновляет цель и создаёт связанную transaction.

07 - Dynamic Flow - Goal Operation via gRPC + Kafka + Worker

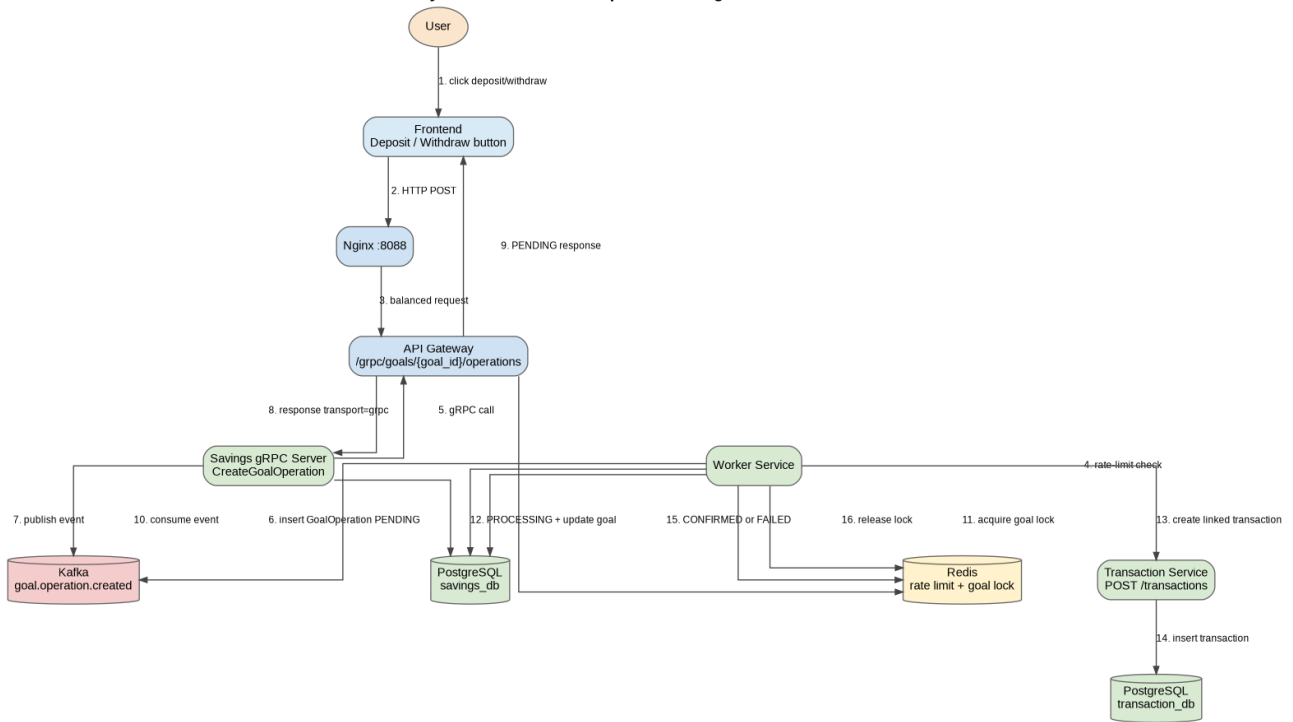


Рисунок - 7. C4 Dynamic - Goal Operation via gRPC + Kafka + Worker

8. Integration Test Coverage View

Это вспомогательная диаграмма, не классическая C4, но она объясняет, что доказывают integration tests: service health, JWT flow, MinIO uploads, gRPC calls, Kafka worker processing и recommendations.

08 - Integration Test Coverage

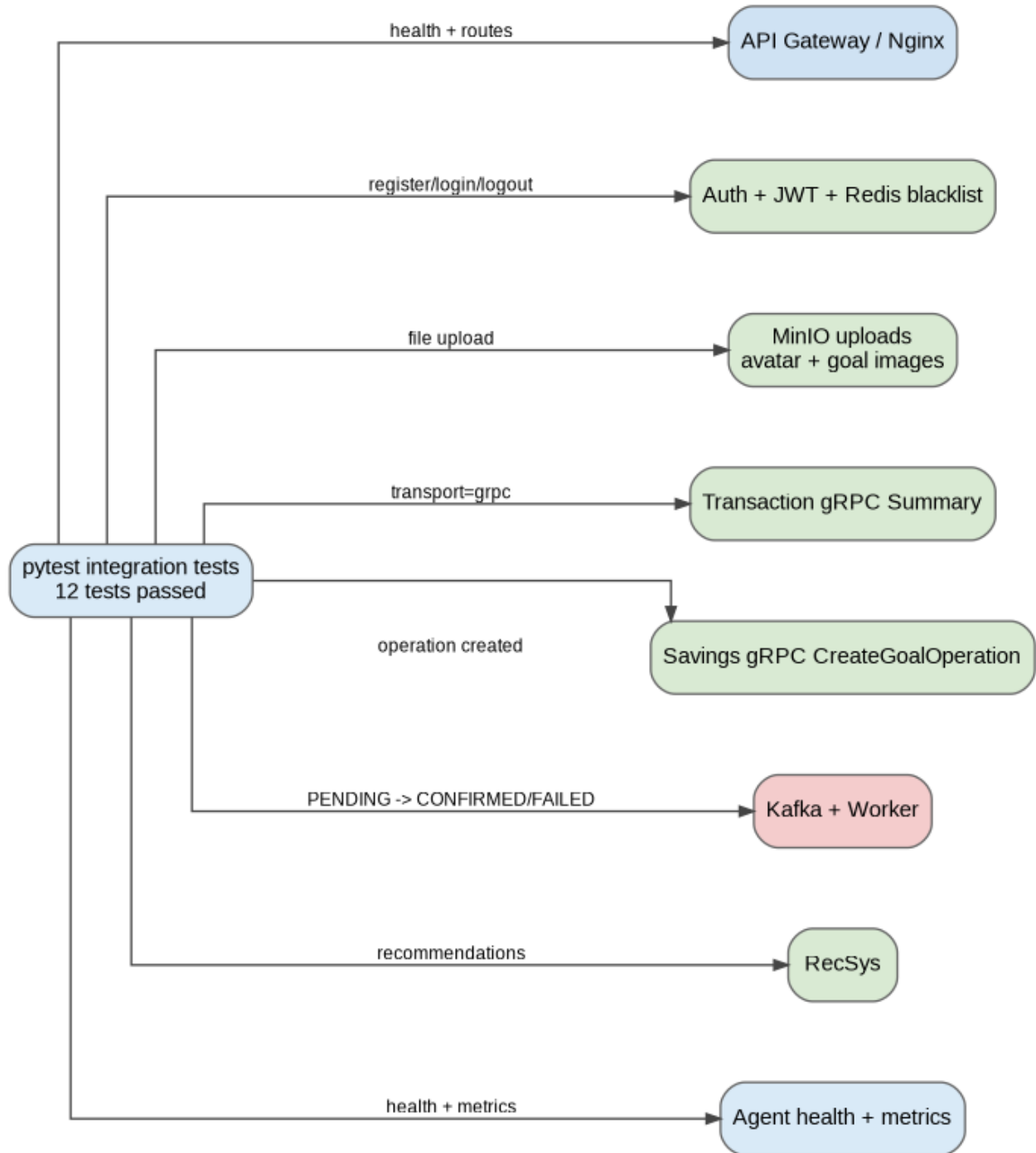


Рисунок - 8. Integration Test Coverage View